TATA CONSULTANCY SERVICES

TATA CONSULTANCY SERVICES

PROJECT REPORT

Title of the Project

Restaurant Order and Menu Simulation System

Submitted By:

Name	:	Hansha Yadav
Enrollment No.	:	0805CS221033
Course	:	B.Tech (Computer Science and Engineering)
Branch	:	Computer Science and Engineering

Submitted To:

Tata Consultancy Services (TCS)

Institute Name:

Jawaharlal Institute of Technology
Borawan

Academic Year:

2022 – 26

Date of Submission:

____/ ___/ _____

TABLE OF CONTENT

ACKNOWLEDGEMENTS

This project titled **Restaurant Order and Menu Simulation System** has been developed as part of an industry-aligned learning initiative provided by **Tata Consultancy Services (TCS)**. The project aims to strengthen practical knowledge of Object-Oriented Programming principles and enterprise-level software design.

I would like to express sincere gratitude to the mentors and trainers associated with the TCS learning initiative for providing structured guidelines, project workflows, and continuous support throughout the development lifecycle.

Special thanks to the academic faculty members of the institution for facilitating the technical infrastructure and learning environment required to complete this project successfully.

This project has significantly contributed to improving practical skills in Java programming, system design, and modular software development aligned with industry expectations.

OBJECTIVE AND SCOPE

Objective

The primary objective of this project is to design and implement a **Restaurant Order and Menu Simulation System** that models real-world restaurant workflows using **Object-Oriented Programming (OOP) principles in Java**, following industry-standard development practices encouraged by **Tata Consultancy Services**.

The system is intended to simulate restaurant operations such as menu creation, customer ordering, billing logic, and discount management through modular and scalable architecture.

Key Objectives

- To implement **industry-grade modular software architecture**
- To apply **Object-Oriented Programming principles** such as:
    - Abstraction
    - Encapsulation
    - Inheritance
    - Polymorphism
- To simulate a **customer-driven ordering workflow**
- To implement **polymorphic billing engines**
- To generate **dynamic invoice outputs**
- To design maintainable and reusable Java components

Scope

The scope of this system extends to simulating a structured restaurant ordering environment that reflects typical operations in small to medium-scale restaurant systems.

The project includes:

- Modular menu management system
- Dynamic order creation
- Quantity-based item handling
- Order lifecycle tracking
- Discount processing
- Bill generation mechanism
- Console-based simulation environment

Industry Relevance Scope

This system can be extended into:

- POS (Point of Sale) Systems
- Restaurant Management Software
- Cloud-based Ordering Systems

- Online Food Delivery Platforms
- Enterprise Resource Planning (ERP) modules

The modular architecture supports scalability, enabling integration with database systems and graphical user interfaces in future versions.

PROBLEM STATEMENT

Modern restaurant operations require reliable, scalable, and automated systems to manage customer orders efficiently while minimizing manual errors.

Traditional manual systems often result in:

- Incorrect bill calculations
- Slow order processing
- Limited scalability
- Lack of real-time order tracking

The core problem addressed in this project is:

To design and implement an industry-aligned restaurant simulation system that automates menu management, customer ordering, billing logic, and discount processing using Object-Oriented Programming techniques in Java.

The system must:

- Support dynamic order creation
- Enable quantity-based ordering
- Process multiple order types
- Apply conditional discounts
- Generate accurate billing outputs
- Maintain structured order lifecycle tracking

This project aligns with enterprise-level design principles followed in professional software development environments.

EXISTING APPROACHES

Before the adoption of automated restaurant systems, several traditional and semi-automated methods were widely used.

1. Manual Order Processing Systems

In traditional restaurant environments:

- Orders were recorded manually on paper
- Billing calculations were performed manually

- No centralized record management existed

Limitations

- High probability of calculation errors
- Increased service time
- Lack of scalability
- No historical data storage
- Inefficient customer service workflow

2. Basic Computerized Billing Systems

Basic desktop billing applications were later introduced to improve accuracy.

These systems typically:

- Store menu data
- Perform arithmetic billing
- Generate printed receipts

Limitations

- Limited modular design
- Poor extensibility
- No support for dynamic workflows
- Lack of structured object-oriented design
- Minimal support for polymorphic billing logic

3. Enterprise Restaurant Systems

Modern enterprise systems integrate:

- POS devices
- Kitchen management systems
- Inventory tracking
- Cloud storage

However, such systems require structured software architecture and professional programming standards.

This project simulates the **foundational architectural model** used in enterprise restaurant systems.

APPROACH / METHODOLOGY

Tools and Technologies Used

The development of this project follows a **modular object-oriented methodology** aligned with software engineering best practices commonly used in enterprise environments such as those promoted by **Tata Consultancy Services**.

Programming Language

Java

Java was selected as the core programming language due to:

- Strong Object-Oriented support
- Platform independence
- Robust package management
- Industry adoption in enterprise applications
- Built-in memory management

Java supports scalable development and is widely used in backend enterprise systems.

Software Development Tools

- **Java Development Kit (JDK)**
- **Integrated Development Environment (IDE)**
  - Eclipse
- **GitHub**
  Used for version control and project repository management.
- **UML Design Tools**
  - Draw.io
  - StarUML

Development Methodology

The project was implemented using the following structured phases:

Phase 1 — System Architecture Design

- Identification of restaurant entities
- Creation of abstract base classes
- UML modeling

Phase 2 — Modular Class Development

- Implementation of menu hierarchy
- Encapsulation of pricing logic
- Factory-based object creation

Phase 3 — Workflow Simulation

- Customer order creation
- Quantity-based item management
- Billing logic implementation

Phase 4 — Testing and Validation

- Unit-level testing
- Workflow simulation testing
- Billing verification

WORKFLOW

The system workflow represents the logical sequence of operations performed during restaurant order processing.

Workflow Description

1. System initializes menu items
2. Menu list is displayed
3. Customer selects order type
4. Customer selects menu items
5. Quantity is entered
6. Discount selection occurs
7. Billing logic is executed
8. Final invoice is generated

Workflow Representation

IMPLEMENTATION

Data Collection

The data used in the **Restaurant Order and Menu Simulation System** is predefined to simulate real-world restaurant operations. Since this project is designed as a simulation system, structured sample data is used instead of live database integration.

The following categories of data were considered:

Menu Data

Menu data includes:

- Item ID
- Item Name
- Category
- Base Cost
- Tax Rate

Sample Menu Dataset

Item ID	Item Name	Category	Base Price (₹)	Tax Rate
1	Spring Rolls	Appetizer	120	5%
2	Paneer Butter Masala	Main Course	250	10%
3	Cold Coffee	Beverage	90	3%

This dataset is loaded into the system during initialization using the **MenuFactory class**, ensuring consistent object creation.

Processing Steps

The internal processing logic of the system follows structured execution steps aligned with modular design principles.

Step-by-Step Processing

1. Initialize predefined menu items
2. Store items in menu list
3. Display menu to customer
4. Accept item selection input
5. Accept quantity input
6. Create OrderItem objects
7. Store items in Order class

8. Apply selected discount
9. Calculate total price
10. Generate bill output

This step-wise processing improves system clarity and debugging efficiency.

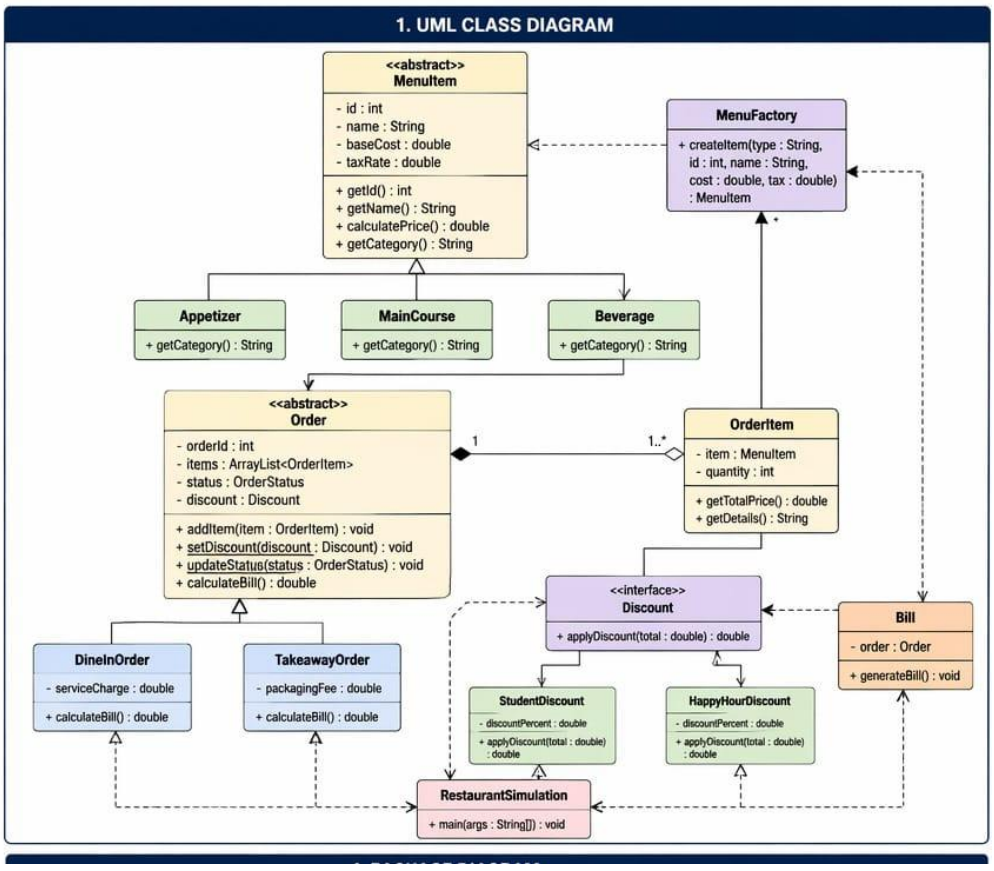
Diagrams — Charts and Tables

The following diagrams are recommended to be included in this section:

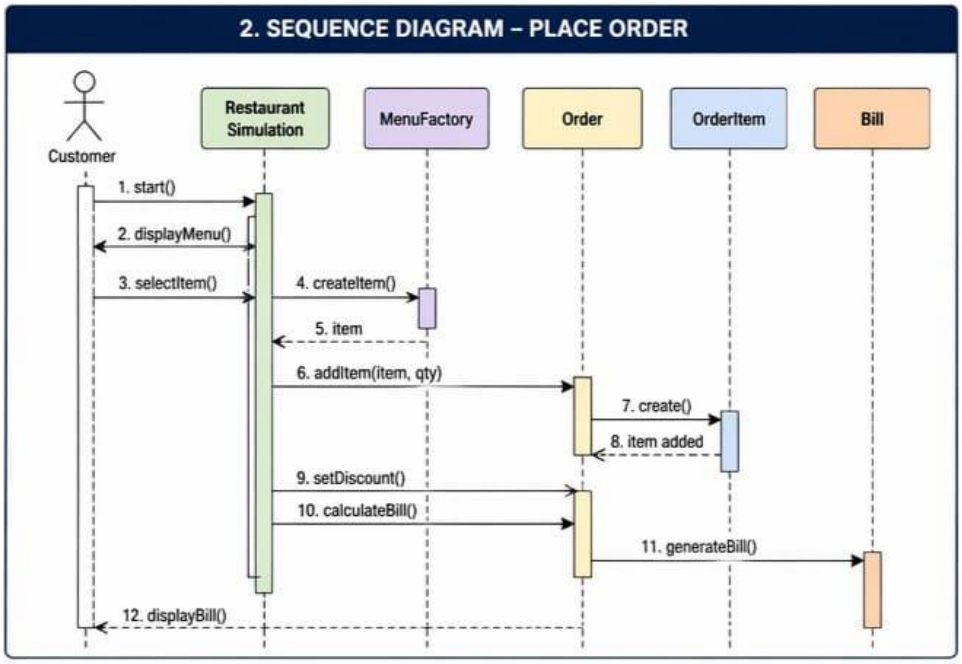
Required Diagrams

You should insert:

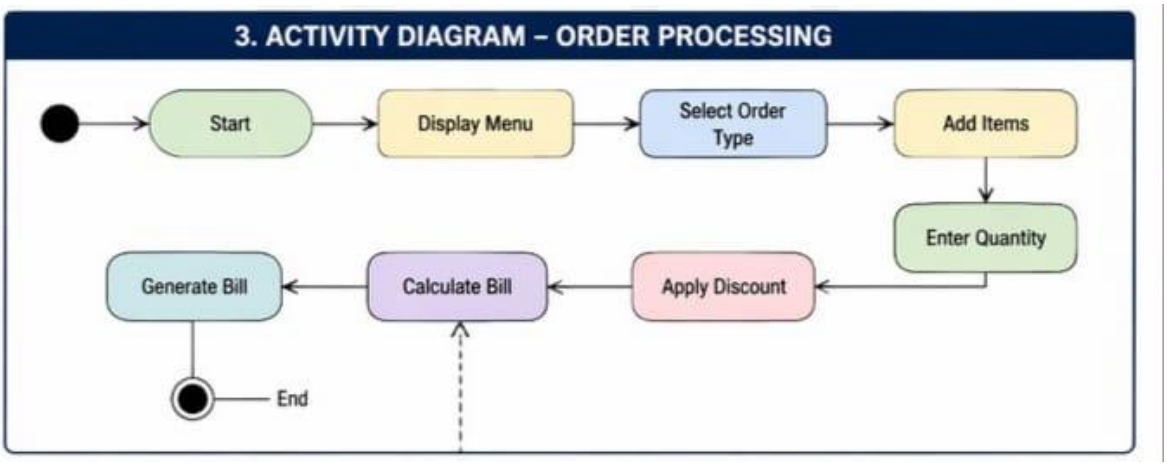
1. UML Class Diagram



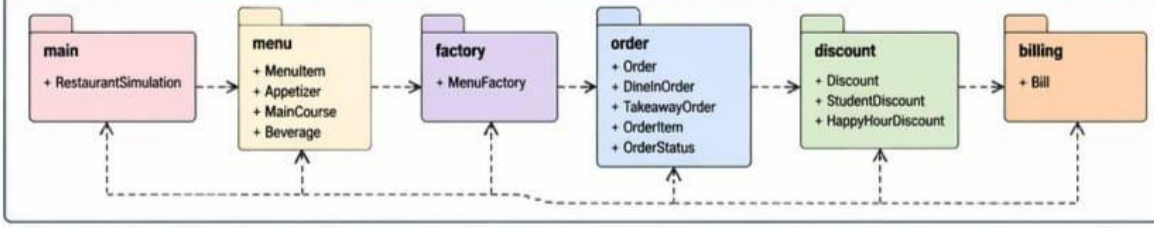
2. SEQUENCE DIAGRAM – PLACE ORDER

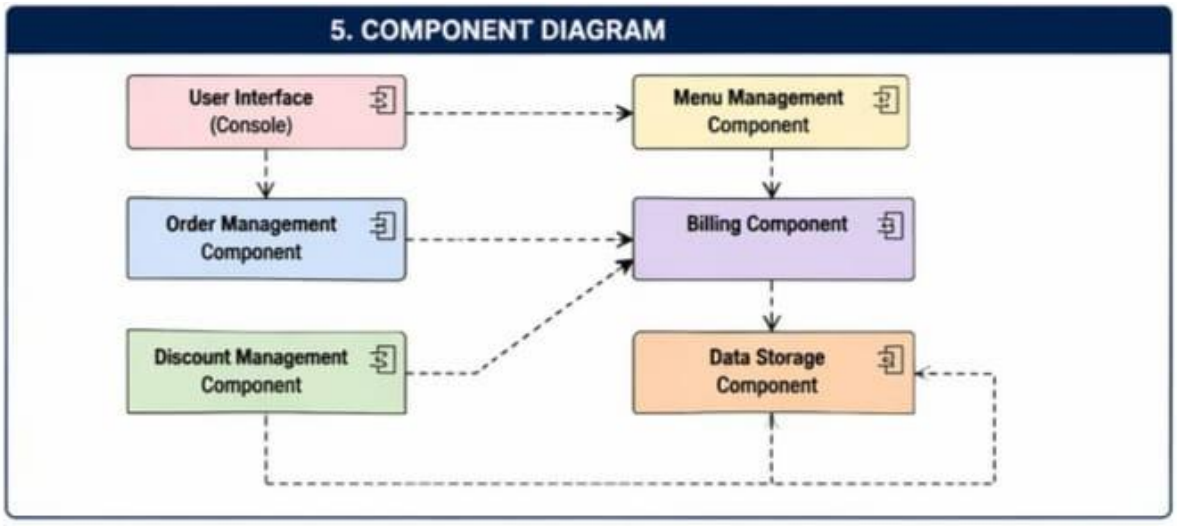


3. ACTIVITY DIAGRAM – ORDER PROCESSING



4. PACKAGE DIAGRAM





2. Package Diagram
3. Order Processing Flow Diagram

Table — Package Responsibilities

Package	Responsibility
menu	Menu hierarchy creation
factory	Menu object creation
order	Order processing logic
discount	Discount handling
billing	Bill generation
main	System execution

SOLUTION DESIGN

The solution is based on a **layered modular architecture** that separates functionality into dedicated packages. This improves maintainability, scalability, and reusability.

Architectural Design Overview

The system architecture is divided into functional modules.

Main Execution Layer
↓
Menu Management Layer

↓
Order Processing Layer
↓
Discount Handling Layer
↓
Billing Layer

Each layer performs specific responsibilities, reducing dependency conflicts.

Class Design Strategy

The system follows structured object-oriented design patterns.

Key Design Components

1. Abstract Classes
2. Interfaces
3. Inheritance
4. Factory Pattern
5. Aggregation Relationships

Key Class Relationships

Inheritance Relationships

MenuItem
↑
Appetizer
MainCourse
Beverage
Order
↑
DineInOrder
TakeawayOrder

Aggregation Relationships

Order contains OrderItem
OrderItem contains MenuItem

Interface Relationships

Discount
↑
StudentDiscount
HappyHourDiscount

This modular design improves system flexibility and scalability.

CHALLENGES & OPPORTUNITIES

During the development of this system, several technical and logical challenges were encountered.

Challenges Faced

1. Designing Abstract Class Hierarchy

Creating a structured class hierarchy while maintaining proper abstraction required careful planning.

Key issue:

- Avoiding direct instantiation of base classes

Solution:

- Used abstract classes and method overriding

2. Implementing Quantity-Based Ordering

Handling multiple quantities required introducing an additional class.

Solution:

- Created `OrderItem` class
- Linked MenuItem with quantity

3. Managing Polymorphic Billing

Different billing rules for different order types required dynamic method execution.

Solution:

- Used method overriding
- Implemented runtime polymorphism

4. Maintaining Data Encapsulation

Preventing unauthorized data modification required strict access control.

Solution:

- Used private variables
- Implemented getters and setters

Opportunities Identified

This project presents several opportunities for future expansion:

- Database integration
- Cloud deployment
- Graphical user interface development
- Real-time order tracking
- Multi-user support

REFLECTIONS ON THE PROJECT

The development of the **Restaurant Order and Menu Simulation System** provided valuable insights into practical software development and system architecture design.

This project enhanced understanding of:

- Real-world system modeling
- Object-Oriented Programming
- Software modularization
- Logical workflow management

Key learning outcomes include:

- Effective use of Java packages
- Implementation of polymorphic behavior
- Design of reusable classes
- Understanding of enterprise-style workflows

The project also improved debugging and testing skills by simulating realistic restaurant scenarios.

RECOMMENDATIONS

Based on the implementation experience, the following recommendations are suggested for further development:

1. Implement database connectivity using **MySQL**
2. Add graphical user interface using **Java Swing**
3. Introduce authentication system
4. Enable order history storage
5. Integrate reporting tools

These improvements would transform the system into a production-ready solution.

OUTCOME / CONCLUSION

The **Restaurant Order and Menu Simulation System** successfully demonstrates the application of object-oriented programming principles in solving real-world workflow challenges.

The project achieved the following outcomes:

- Developed a structured restaurant simulation system
- Implemented modular software architecture
- Supported quantity-based ordering
- Applied dynamic discount logic
- Generated accurate billing output

The system demonstrates how enterprise-level workflows can be simulated using structured programming techniques. The modular design ensures that the system remains scalable and maintainable for future upgrades.

Overall, this project meets the defined objectives and reflects industry-aligned development practices.

ENHANCEMENT SCOPE

The system can be enhanced by integrating additional features to improve performance and usability.

Future Enhancement Possibilities

1. Database Integration

Use:

- MySQL
- MongoDB

To store:

- Menu data
- Order history
- Customer records

2. GUI-Based System

Convert console system into graphical interface using:

- Java Swing
- JavaFX

3. Multi-User Support

Enable:

- Multiple customers
- Concurrent order handling

4. Online Ordering Integration

Connect system with:

- Web applications
- Mobile applications

5. Payment Integration

Add:

- Digital payment support
- Invoice printing

These enhancements will significantly increase system usability.

RESEARCH QUESTIONS AND RESPONSES

This section reflects technical learning outcomes derived during development.

Question 1

Why is abstraction important in object-oriented systems?

Response

Abstraction helps in hiding implementation details and exposing only relevant functionalities. In this project, the `MenuItem` class is declared abstract to prevent direct instantiation and enforce subclass specialization.

Question 2

How does polymorphism improve system flexibility?

Response

Polymorphism allows methods to behave differently based on object types. In this system, the `calculateBill()` method behaves differently for `DineInOrder` and `TakeawayOrder`.

Question 3

What role does encapsulation play in software security?

Response

Encapsulation protects sensitive data from unauthorized access. Private variables ensure controlled data access through getter methods.

Question 4

Why was Factory Pattern used?

Response

Factory Pattern simplifies object creation and improves code reusability. It also allows centralized management of menu item instantiation.

REFERENCES

The following references were used during project development:

1. Oracle Java Documentation
   https://docs.oracle.com/javase/
2. Object-Oriented Programming Concepts
   Herbert Schildt — Java Programming Guide
3. UML Modeling Reference
   https://www.uml.org
4. Industry-aligned project guidelines provided under the **TCS learning initiative**